

KEYSTONE Frontend

A React + TypeScript SPA for the KEYSTONE field-service platform, built against the **current** state of the **DeliveryService** Spring Boot backend — JWT auth plus User and Customer management. It is not the full KEYSTONE feature set from the project brief (no Site, WorkOrder, Part, or TimeLog yet — those endpoints don't exist on the backend yet). Every screen that would depend on them shows an explicit "not built yet" panel instead of fake data.

What's here

- **Login / Register** — `POST /api/users/login` and `POST /api/users`, JWT stored client-side, attached to every request via an axios interceptor.
- **Role-aware routing** — one `/dashboard` route that renders a different view for **MANAGER**, **DISPATCHER**, **TECHNICIAN**, and **CUSTOMER**. Routes are also gated by role client-side (`ProtectedRoute`), matching what the backend actually restricts.
- **Users page** (`/users`) — list, search, create, delete. Backend-gated to **MANAGER** *in the UI only* — see note below.
- **Customers page** (`/customers`) — paginated, searchable list plus create/edit, matching `GET/POST/PUT /api/customers`. Gated to **DISPATCHER/MANAGER**, which the backend does enforce via `@PreAuthorize`.
- **Dashboards** — real stat tiles computed from live data (user counts by role, customer count) for Manager/Dispatcher; a profile card for Technician/Customer, since there's no work-order data yet to show them.

Known gap between this UI and the backend

`GET/PUT/DELETE /api/users/**` currently only requires being logged in — **any** role can list, edit, or delete any user, not just Manager. The Users page is hidden from non-Manager roles in the nav and gated in the router, but that's a UI convenience, not enforcement (the brief is explicit about this: "hiding a control in React is a usability choice, not a security control"). If you want this locked down properly, add `@PreAuthorize("hasRole('MANAGER')")` to the mutating `UserController` endpoints on the backend.

Similarly, self-registration (`POST /api/users`) lets a new account pick any role, including **MANAGER** — there's no invite/approval flow. Fine for development; worth gating before this goes anywhere real.

Running it

```
npm install
cp .env.example .env    # adjust VITE_API_BASE_URL if your backend isn't on
                          :8080
npm run dev
```

The backend's CORS config (`SecurityConfig.corsConfigurationSource`) allows any `http://localhost:*` / `http://127.0.0.1:*` origin, so the default Vite port (5173) works without extra configuration.

Project layout

```
src/  
  api/           axios client + one module per backend resource  
  components/    shared UI (DataTable, Navbar, ProtectedRoute, ...)  
  context/       AuthContext – login/register/logout, JWT persistence  
  pages/         route-level screens  
  pages/dashboards/ one component per role, switched by DashboardRouter  
  types.ts       TypeScript mirrors of the backend's DTOs
```